

Выполнил: Цыплаков Станислав Рустамович, Р4109

Оборудование: Ноутбук и android смартфон

Софт для захвата трафика – Proxyman

Часть 1. Разведка API через Swagger

Method	Path (http:// 82.39.214.8: 3001)	Назначение	Параметры	Пример запроса	Пример ответа
POST	/cats/search	Поиск по имени и доп. характеристикам	Поиск по имени	<pre>{ "name": "Коте", "gender": "male" }</pre>	<pre>{ "groups": [{ "title": "К", "cats": [{ "id": 12, "name": "Коте", "description": "Котэ", "tags": null, "gender": "male", "likes": 0, "dislikes": 0 }], "count": 1 }], "count": 1 }</pre>

GET	/cats/get-by-id/{id}	Получение кота по id	id	Запрос передается в адресной строке	<pre>{ "cat": { "id": 5, "name": "Буся", "description": "Запись для ручного редактирования", "tags": null, "gender": "female", "likes": 3, "dislikes": 2 } }</pre>
POST	/cats/add	Добавление списка имен	Имя, пол, описание	<pre>{ "cats": [{ "name": "Шанти", "gender": "female", "description": "Черная бестия" }] }</pre>	<pre>{ "cats": [{ "id": 19, "name": "Шанти", "description": "Черная бестия", "tags": null, "gender": "female", "likes": 0, "dislikes": 0 }] }</pre>

POST	/cats/save-description	Сохранение описания имени		{ "catId": 19, "catDescription": "Ночной тыгыдык" }	{ "cat": { "id": 19, "name": "Шанти", "description": "Ночной тыгыдык", "tags": null, "gender": "female", "likes": 0, "dislikes": 0 } }
POST	/cats/{catId}/like	Добавление лайка коту	id	19	OK
DELETE	/cats/{catId}/like	Удалить лайк у кота	id	19	OK
POST	/cats/{catId}/dislike	Добавление дизлайка коту	id	19	OK
DELETE	/cats/{catId}/dislike	Удалить дизлайк	id	19	OK

GET	/cats/likes-rating	Получение топа 10 лайканых	Нет	Пусто	[{ "name": "Буся", "likes": 3 }, { "name": "Котэ", "likes": 3 }, { "name": "Кошка", "likes": 1 }, { "name": "Вел", "likes": 0 }, { "name": "Кекс", "likes": 0 }, { "name": "Коте", "likes": 0 }, { "name": "Ллл", "likes": 0 }, { "name": "Снежок", "likes": 0 }, { "name": "Тест",
-----	--------------------	----------------------------------	-----	-------	---

					<pre> "likes": 0 }, { "name": "Tom", "likes": 0 }]</pre>
--	--	--	--	--	--

GET	/cats/dislikes-rating	Получение топа 10 дизлайканых	Нет	Пусто	[{ "name": "Буся", "dislikes": 2 }, { "name": "Вел", "dislikes": 0 }, { "name": "Кекс", "dislikes": 0 }, { "name": "Коте", "dislikes": 0 }, { "name": "Котэ", "dislikes": 0 }, { "name": "Кошка", "dislikes": 0 }, { "name": "ЛЛЛ", "dislikes": 0 }, { "name": "Снежок",
-----	-----------------------	-------------------------------------	-----	-------	--

					<pre> "dislikes": 0 }, { "name": "Тест", "dislikes": 0 }, { "name": "Том", "dislikes": 0 }] </pre>
GET	/cats/{catId} /photos	Получение изображений по id кота	Id кота	12	<pre> { "images": ["/photos/ima ge- 178251198168 8.jpg"] } </pre>
POST	/cats/{catId} /upload	Добавление изображения кота	Id кота и изображ ение	19 булочка.jpg	<pre> { "fileUrl": "/photos/ima ge- 178252613331 1.jpg" } </pre>

DELETE	/cats/{catId}/remove	Удаление кота	Id кота	19	<pre>{ "data": null, "isBoom": true, "isServer": false, "output": { "statusCode" : 404, "payload": { "statusCode" : 404, "error": "Not Found", "message": "Не найден id кота" }, "headers": {} } }</pre>
--------	----------------------	---------------	---------	----	---

Часть 2. Карта действий приложения

Включите прокси и выполните в мобильном приложении минимум следующие действия:

1. Поиск котика.
2. Открытие карточки.
3. Лайк.
4. Дизлайк.
5. Добавление нового котика.
6. Изменение описания.
7. Загрузка изображения.
8. Открытие рейтинга.

Действие в приложении	HTTP Method	URL http://82.39.214.8:3001	Request Body	Response code	Response body
Поиск котика	POST	/api/core/cats/search	{ "gender": null, "name": "буся", "order": "asc" }	200	{ "groups": [{ "title": "Б", "cats": [{ "id": 5, "name": "Буся", "description": "Запись для ручного редактирования", "tags": "", "gender": "female", "likes": 3, "dislikes": 2 }], "count": 1 }] }
Открытие карточки	GET	/api/core/cats/get-by-id?id=5	Empty Body	200	{ "cat": { "id": 5, "name": "Буся", "description": "Запись для ручного редактирования", "tags": "", "gender": "female", "likes": 3, "dislikes": 2 } }
Лайк	POST	/api/likes/cats/5/likes	{ "dislike": false, "like": true }	200	{ "id": 5, "name": "Буся", "description": "Запись для ручного редактирования", "tags": "", "gender": "female", "likes": 4, "dislikes": 2 }
Дизлайк	POST	/api/likes/cats/5/likes	{ "dislike": true, "like": false }	200	{ "id": 5, "name": "Буся", "description": "Запись для ручного редактирования", "tags": "", "gender": "female", "likes": 4, "dislikes": 3 }

Добавление нового котика	POST	/api/core/cats/add	{ "cats": [{ "description": " ", "gender": "female", "name": "Шанти" }] }	200	{ "cats": [{ "id": 20, "name": "Шанти", "description": "", "tags": "", "gender": "female", "likes": 0, "dislikes": 0 }] }
Изменение описания	POST	/api/core/cats/save-description	{ "catDescription": "ночной тыгыдык", "catId": 20 }	200	{ "id": 20, "name": "Шанти", "description": "ночной тыгыдык", "tags": "", "gender": "female", "likes": 0, "dislikes": 0 }
Загрузка изображения	GET	/photos/image-1782526575102.jpg	Empty Body	200	Response Header Body Raw - JPEG ↕  Show Inspector 2256 × 4000 pixels • 3.04 MB
Открытие рейтинга	GET	/api/likes/cats/rating	Empty Body	200	{ "likes": [{ "id": 5, "name": "Буся", "description": "Запись для ручного редактирования", "tags": "", "gender": "female", "likes": 4, "dislikes": 3 }, { "id": 13, "name": "Котэ", "description": "Котэк", "tags": "", "gender": "male", "likes": 3, "dislikes": 0 }], }

					<pre>"id": 18, "name": "Кошка", "description": "test' OR '1'='1", "tags": "", "gender": "male", "likes": 1, "dislikes": 0 }, { "id": 8, "name": "Вел", "description": "string", "tags": "", "gender": "male", "likes": 0, "dislikes": 0 }, { "id": 4, "name": "Кекс", "description": "Имя для экспериментов с поиском", "tags": "", "gender": "unisex", "likes": 0, "dislikes": 0 }, { "id": 12, "name": "Коте", "description": "Котэ", "tags": "", "gender": "male", "likes": 0, "dislikes": 0 }, { "id": 11, "name": "Ллл", "description": "oooo", "tags": "", "gender": "male", "likes": 0, "dislikes": 0 }, { "id": 3, "name": "Снежок", "description": "Подходит для проверки лайков", "tags": "", "gender": "male", "likes": 0, "dislikes": 0 }, { "id": 10, "name": "Тест", "description": "", "tags": "",</pre>
--	--	--	--	--	---

					<pre>"gender": "unisex", "likes": 0, "dislikes": 0 }, { "id": 20, "name": "Шанти", "description": "ночной тыгыдык", "tags": "", "gender": "female", "likes": 0, "dislikes": 0 }], "dislikes": [{ "id": 5, "name": "Буся", "description": "Запись для ручного редактирования", "tags": "", "gender": "female", "likes": 4, "dislikes": 3 }, { "id": 8, "name": "Вел", "description": "string", "tags": "", "gender": "male", "likes": 0, "dislikes": 0 }, { "id": 4, "name": "Кекс", "description": "Имя для экспериментов с поиском", "tags": "", "gender": "unisex", "likes": 0, "dislikes": 0 }, { "id": 12, "name": "Коте", "description": "Котэ", "tags": "", "gender": "male", "likes": 0, "dislikes": 0 }, { "id": 13, "name": "Котэ", "description": "Котэк", "tags": "", "gender": "male", "likes": 3,</pre>
--	--	--	--	--	--

					<pre>"dislikes": 0 }, { "id": 18, "name": "Кошка", "description": "test' OR '1'='1", "tags": "", "gender": "male", "likes": 1, "dislikes": 0 }, { "id": 11, "name": "Лл", "description": "oooo", "tags": "", "gender": "male", "likes": 0, "dislikes": 0 }, { "id": 3, "name": "Снежок", "description": "Подходит для проверки лайков", "tags": "", "gender": "male", "likes": 0, "dislikes": 0 }, { "id": 10, "name": "Тест", "description": "", "tags": "", "gender": "unisex", "likes": 0, "dislikes": 0 }, { "id": 20, "name": "Шанти", "description": "ночной тыгыдык", "tags": "", "gender": "female", "likes": 0, "dislikes": 0 }] }</pre>
--	--	--	--	--	--

Часть 3. Изменение запросов

Выполните минимум 6 экспериментов с изменением запросов через прокси, Postman/Insomnia/curl или replay:

1. Измените catId в URL карточки, лайка или дизлайка.
2. Повторите один и тот же запрос лайка несколько раз.
3. Измените описание записи, которую вы не создавали.
4. Попробуйте добавить дубликат имени.
5. Отправьте невалидные значения gender, name, catDescription.
6. Попробуйте вызвать web API-методы, которых нет в основном мобильном сценарии.
7. Проверьте, что происходит при обращении к несуществующему catId.
8. Проверьте загрузку файла: корректное изображение, файл с неправильным расширением, файл с неправильным MIME type.

Для каждого эксперимента зафиксируйте:

1. исходный запрос;
2. изменённый запрос;
3. ответ сервера;
4. изменилось ли состояние данных;
5. является ли поведение багом или security-проблемой;
6. как бы вы исправили проблему.

Исходный запрос	Измененный запрос	Ответ сервера	Изменилось ли состояние данных?	Является ли это багом или security-проблемой?	Как бы я исправил проблему?
http://82.39.214.8:3001/api/core/cats/get-by-id?id=5	http://82.39.214.8:3001/api/core/cats/get-by-id?id=999999	404	Нет, GET запрос идиempotent и на данные не влияет	Нет	Не трогал бы
http://82.39.214.8:3001	http://82.39.214.8:3001	{ "id": 13, "name": "Котэ",	Да, количество	Да, это является	Ввел бы учетные

/api/likes/cats/13/likes	/api/likes/cats/13/likes	<pre>"description": "Котэк", "tags": "", "gender": "male", "likes": 5, "dislikes": 0 }</pre>	лайков увеличилось на два, а не на один	уязвимость ю, ибо нет проверки постановки лайка	записи, по которым закреплялся бы лайк за каждым пользователем
http://82.39.214.8:3001/api/core/cats/save-description <pre>{ "catDescription": "Котэк", "catId": 13 }</pre>	http://82.39.214.8:3001/api/core/cats/save-description <pre>{ "catDescription": "Алькотек", "catId": 13 }</pre>	<pre>{ "id": 13, "name": "Котэ", "description": "Алькотек", "tags": "", "gender": "male", "likes": 6, "dislikes": 0 }</pre>	Да, подменило описание карточки с котиком	Это уязвимость безопасности, ибо нет проверки авторизации на данную операцию	Ввести проверку прав пользователя на редактирование карточки
http://82.39.214.8:3001/api/core/cats/add <pre>{ "cats": [{ "description": "ppp", "gender": "male", "name": "ppp" }] }</pre>	http://82.39.214.8:3001/api/core/cats/add <pre>{ "cats": [{ "description": "ppp", "gender": "male", "name": "ppp" }] }</pre>	STATUS CODE: 200 OK <pre>{ "cats": [{ "errorDescription": "Такое значение уже существует" }] }</pre>	Нет, исключение на дубликаты со стороны сервера отработало штатно	Есть баг с неправильным статус кодом. 200 тут не к месту	Нужно возвращать 4xx код
http://82.39.214.8:3001/api/core/cats/add <pre>{ "cats": [{ "description": "ppp", "gender": "female", "name": "эээ" }] }</pre>	http://82.39.214.8:3001/api/core/cats/add <pre>{ "cats": [{ "errorDescription": "Цифры не принимаются!" }] }</pre>	<pre>{ "cats": [{ "errorDescription": "Цифры не принимаются!" }] }</pre>	Нет, исключение на невалидные со стороны сервера отработало штатно	Есть баг с неправильным статус кодом. 200 тут не к месту	Нужно возвращать 4xx код
http://82.39.214.8:3001/api/likes/cats/rating	http://82.39.214.8:3001/api/likes/cats/rating	404	Нет, запрос индепотентен	Есть уязвимость безопасности. Приложение не должно работать с веб api	Ввести дополнительную проверку клиента, с которого происходит, входит + должна возвращаться 400 с пояснением

					ответа на запрос
--	--	--	--	--	---------------------

Часть 4. Изменение ответов

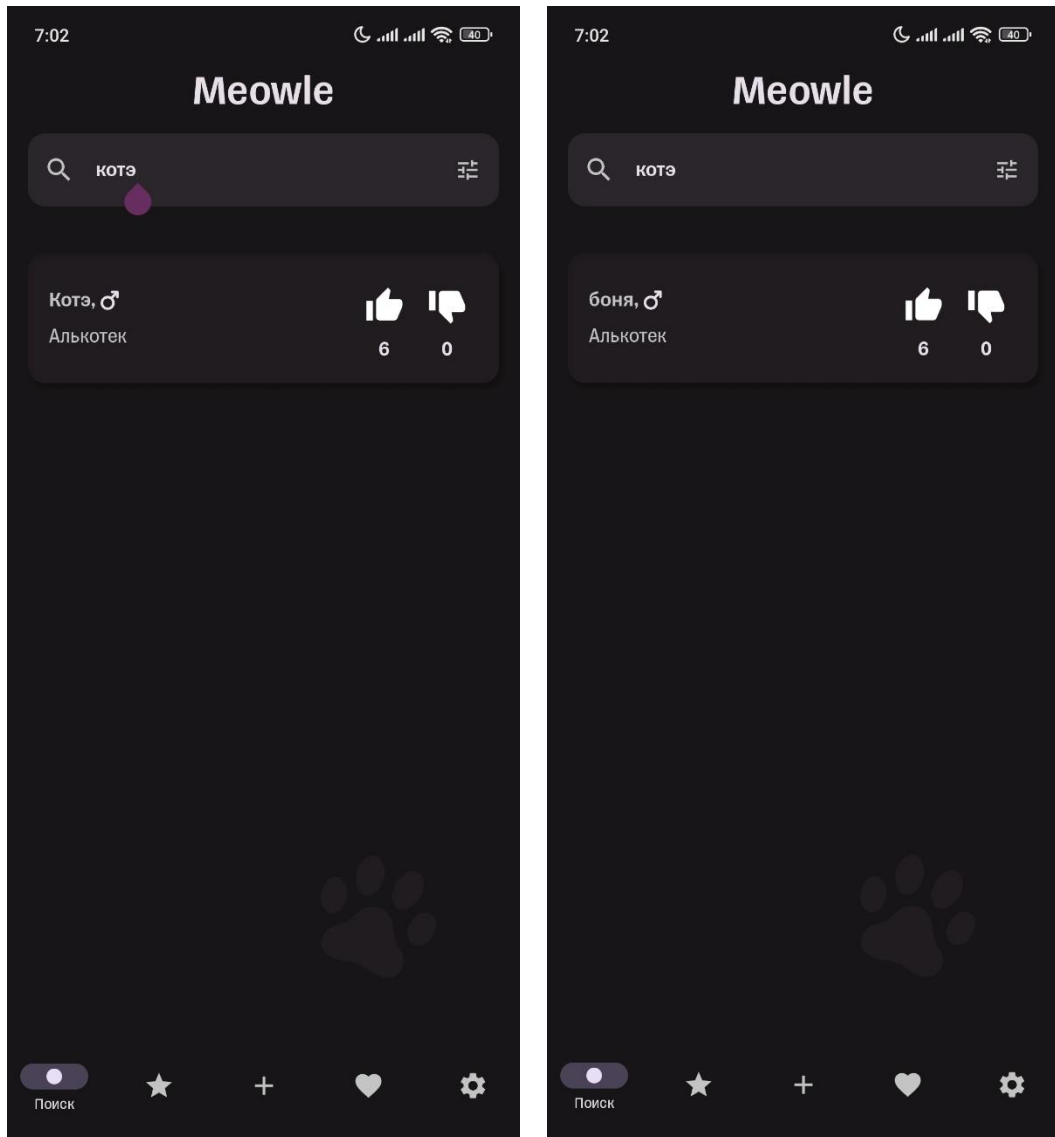
Через Charles Rewrite/Map Local или аналог в Proxuman измените ответы сервера:

- 1. Подмените имя или описание котика в ответе поиска.
- 2. Подмените количество лайков/дизлайков.
- 3. Подмените список результатов поиска.
- 4. Подмените список фото или URL изображения.

Сделайте скриншоты до и после. Объясните, какие изменения остались только на клиенте, а какие реально сохранились на сервере.

1. Подмена имени котика в ответе поиска

До и после:



POST 200 http://82.39.214.8:3001/api/core/cats/search									
Request Header Query Body Raw +					Response Header Body Raw +				
JSON ⌵					JSON ⌵				
<pre>1 { 2 "gender": null, 3 "name": "котэ", 4 "order": "asc" 5 }</pre>					<pre>1 { 2 "groups": [3 { 4 "title": "К", 5 "cats": [6 { 7 "id": 13, 8 "name": "Котэ", 9 "description": "Алькотек", 10 "tags": "", 11 "gender": "male", 12 "likes": 6, 13 "dislikes": 0 14 } 15], 16 "count": 1 17 } 18] 19 }</pre>				

POST 200 http://82.39.214.8:3001/api/core/cats/search									
Request					Response				
Header	Query	Body	Raw	+	Header	Body	Raw	+	JSON
<pre> 1 { 2 "gender": null, 3 "name": "котэ", 4 "order": "asc" 5 } </pre>					<pre> 1 { 2 "groups": [3 { 4 "title": "К", 5 "cats": [6 { 7 "id": 13, 8 "name": "бона", 9 "description": "Алькотек", 10 "tags": "", 11 "gender": "male", 12 "likes": 6, 13 "dislikes": 0 14 } 15], 16 "count": 1 </pre>				

Изменения остались только на клиенте

2. Подмена количества лайков

Фото до и после

Request

Header

Query

Body

Raw

+

Key	Value
Content-Type	application/json; charset=UTF-8
Content-Length	47
Host	82.39.214.8:3001
Connection	Keep-Alive
Accept-Encoding	gzip
User-Agent	okhttp/4.12.0

Response

Header

Body

Raw

+

JSON

⌵

⌵

1	{
2	"groups": [
3	{
4	"title": "К",
5	"cats": [
6	{
7	"id": 13,
8	"name": "Котэ",
9	"description": "Алькотек",
10	"tags": "",
11	"gender": "male",
12	"likes": 6,
13	"dislikes": 0
14	}
15	],
16	"count": 1

Clear

Filter

Auto Select

1/9 rows selected

• 687 MB ↑ 0 B/s ↓ 0 B/s

Scripting

Breakpoint

Request

Header

Query

Body

Raw

+

Key	Value
Content-Type	application/json; charset=UTF-8
Content-Length	47
Host	82.39.214.8:3001
Connection	Keep-Alive
Accept-Encoding	gzip
User-Agent	okhttp/4.12.0

Response

Header

Body

Raw

+

JSON

⌵

⌵

1	{
2	"groups": [
3	{
4	"title": "К",
5	"cats": [
6	{
7	"id": 13,
8	"name": "Котэ",
9	"description": "Алькотек",
10	"tags": "",
11	"gender": "male",
12	"likes": 3236,
13	"dislikes": 0
14	}
15	],
16	"count": 1

Clear

Filter

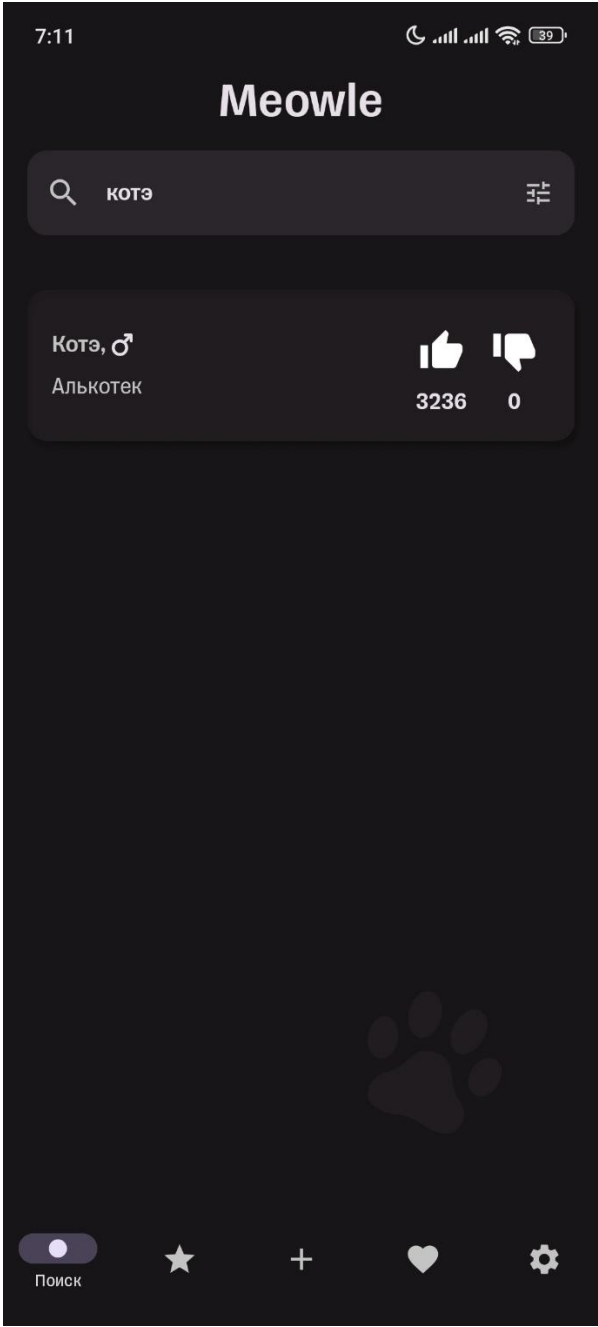
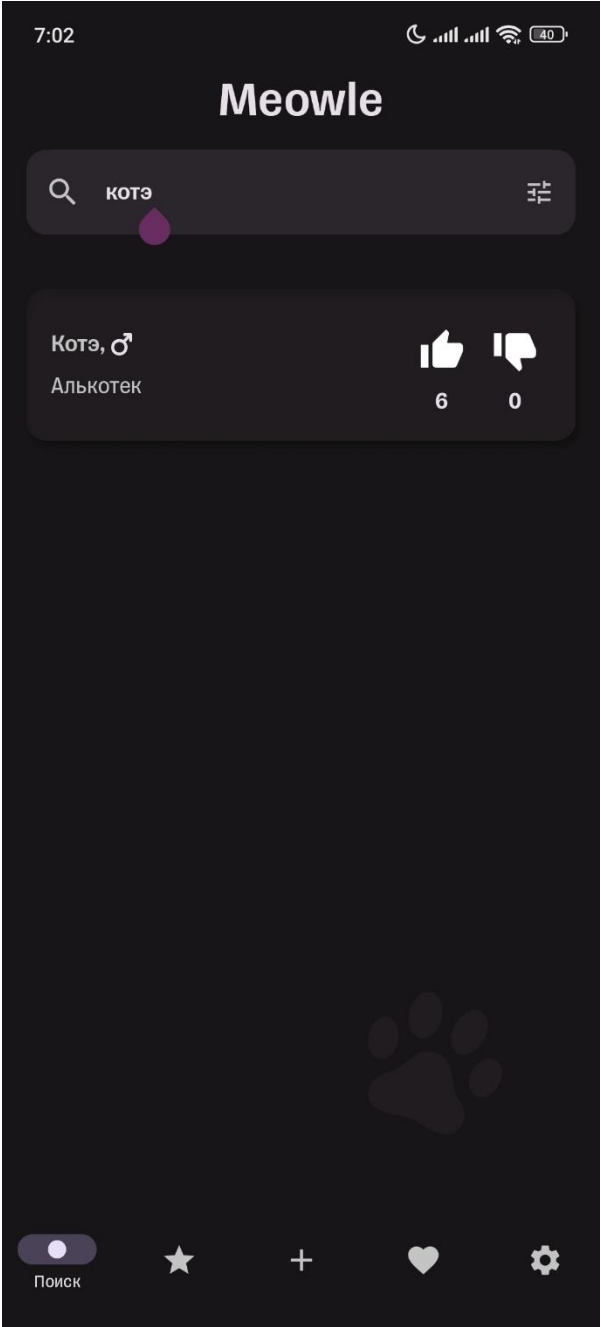
Auto Select

1/8 rows selected

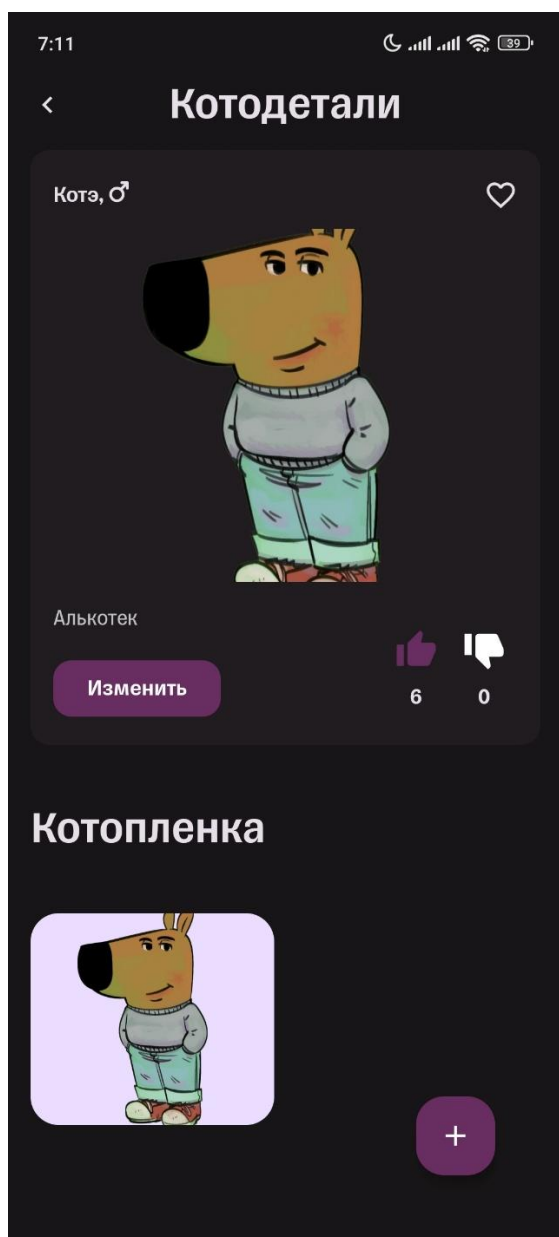
• 683 MB ↑ 0 B/s ↓ 0 B/s

Scripting

Breakpoint



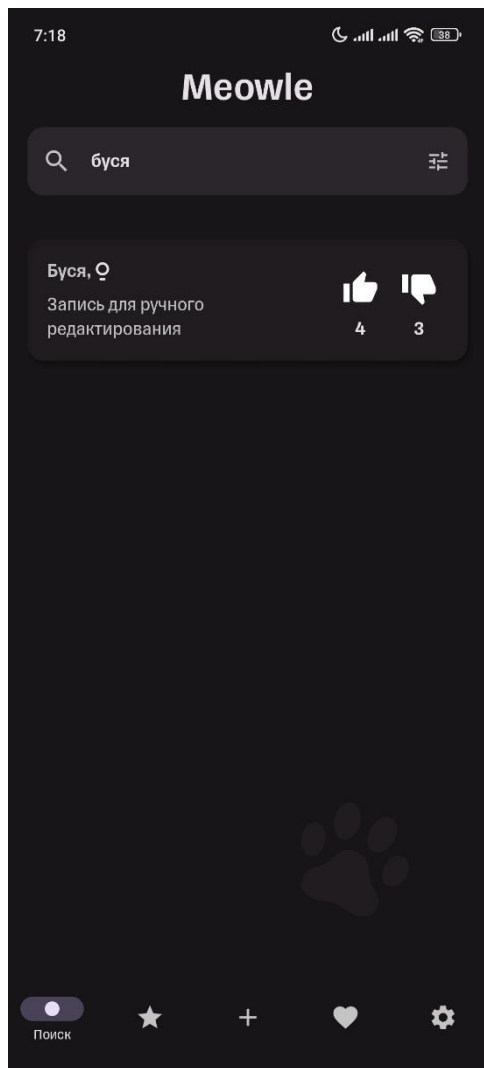
Результат перехода на информацию о коте из результатов подменной выдачи. Можно заметить, что ничего не поменялось



3. Подмените список результатов поиска.

До подмены

POST 200 http://82.39.214.8:3001/api/core/cats/search	
Request	Response
<pre> 1 { 2 "gender": null, 3 "name": "буса", 4 "order": "asc" 5 } </pre>	<pre> 1 { 2 "groups": [3 { 4 "title": "Б", 5 "cats": [6 { 7 "id": 5, 8 "name": "Буса", 9 "description": "Запись для ручного редактирования", 10 "tags": "", 11 "gender": "female", 12 "likes": 4, 13 "dislikes": 3 14 } 15], 16 "count": 1 17 } 18] 19 } </pre>



После подмены:

POST 200 http://82.39.214.8:3001/api/core/cats/search									
Request Header Query Body Raw +					JSON ⌵				
1	2	3	4	5					
{									
"gender": null,									
"name": "буса",									
"order": "asc"									
}									
					Response Header Body Raw +				
					JSON ⌵				
					1	2	3	4	5
					{				
					"groups": [				
					{				
					"title": "К",				
					"cats": [				
					{				
					"id": 4,				
					"name": "Кекс",				
					"description": "Имя для экспериментов с поиском",				
					"tags": "",				
					"gender": "unisex",				
					"likes": 0,				
					"dislikes": 0				
					},				
					{				
					"id": 12,				
					"name": "Коте",				
					"description": "Котэ",				
					"tags": "",				
					"gender": "male",				
					"likes": 0,				
					"dislikes": 0				
					},				
					{				

7:17

38

Meowle

буса

Кекс, ♂

Имя для экспериментов с поиском

00

Коте, ♂

Котэ

00

Котэ, ♂

Алькотек

60

Кошка, ♂

test' OR '1'='1

10

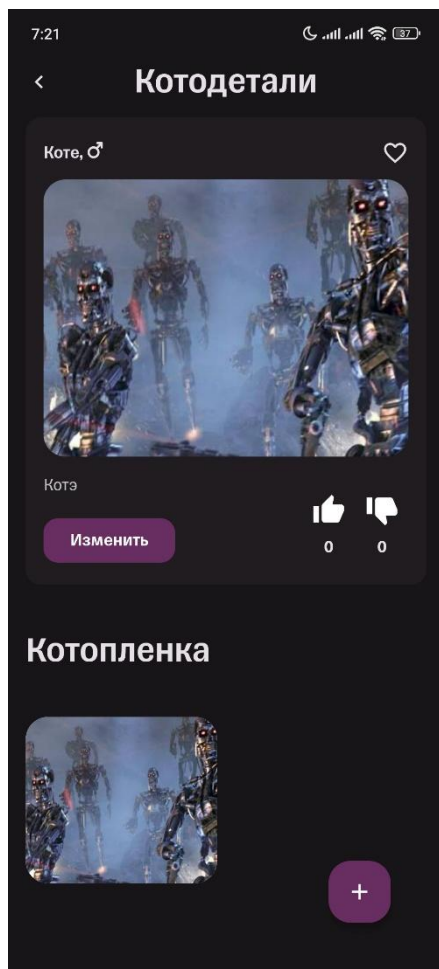
Снежок, ♂

Подходит для проверки лайков

00

Поиск

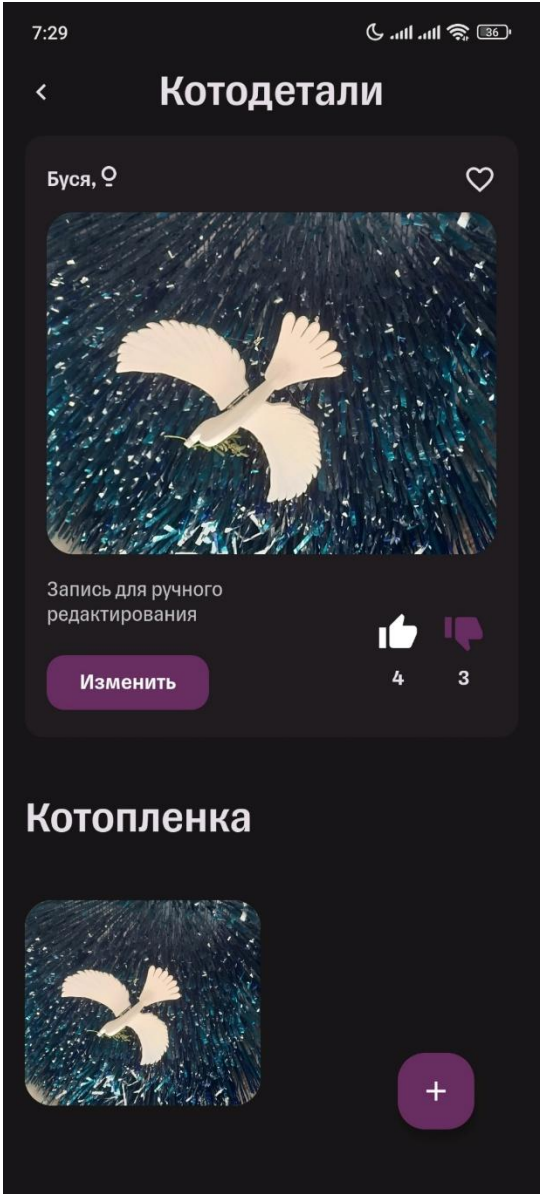
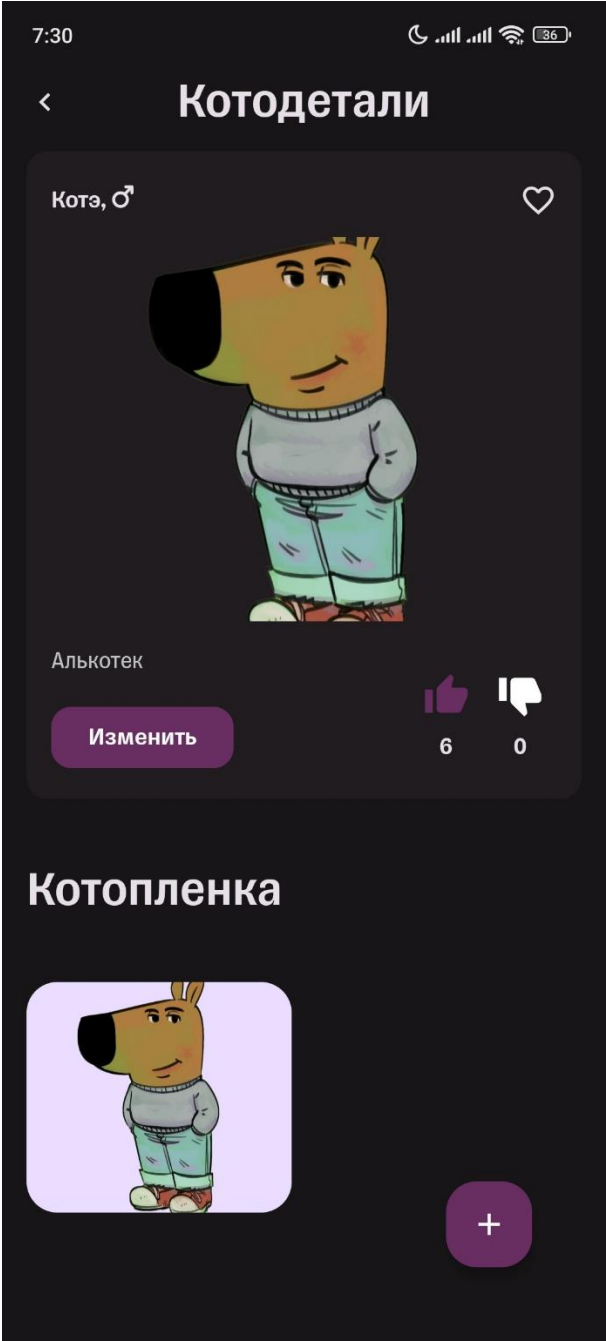
При попытке открыть коте идет перенаправление на коте



На данные сервера такая подмена никак не сыграла

4. Подмените список фото или URL изображения.

До подмены

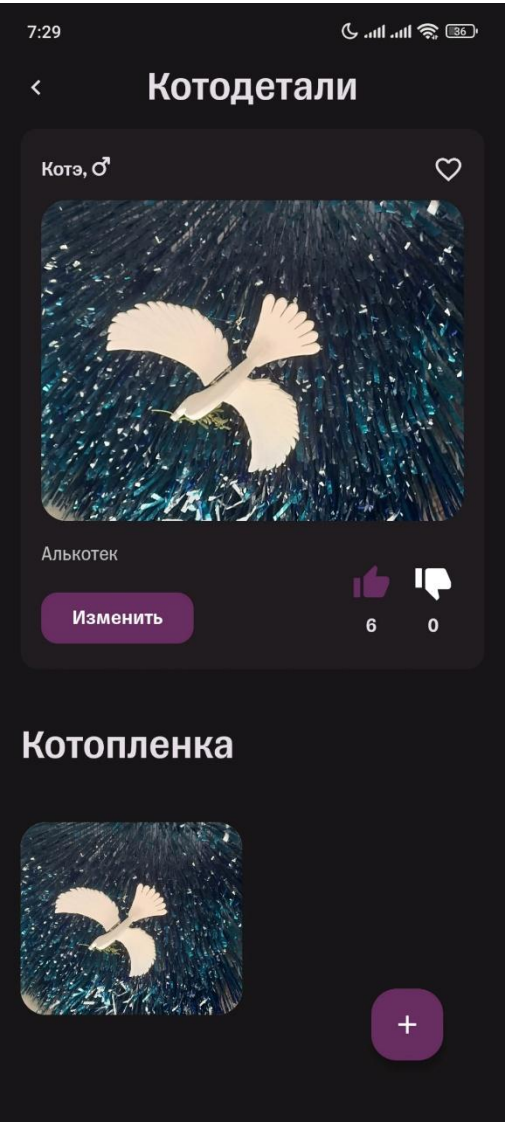


Подменять мы будем фото кота с индексом 13 на фото из индекса 5

GET 200 http://82.39.214.8:3001/api/photos/cats/13/photos	
Request	Response
Header	Header
Query	Body
Body	Raw
Raw	+
+	
Key	Value
Host	82.39.214.8:3001
Connection	Keep-Alive
Accept-Encoding	gzip
User-Agent	okhttp/4.12.0
1 { 2 "images": [3 "/photos/image-1782508887801.jpg" 4] 5 }	

GET 200 http://82.39.214.8:3001/api/photos/cats/5/photos	
Request	Response
Header	Header
Query	Body
Body	Raw
Raw	+
+	
Key	Value
Host	82.39.214.8:3001
Connection	Keep-Alive
Accept-Encoding	gzip
User-Agent	okhttp/4.12.0
1 { 2 "images": [3 "/photos/image-1782534246385.jpg" 4] 5 }	

Результат после подмены:



На стороне Proxuman

GET 200 http://82.39.214.8:3001/api/photos/cats/13/photos	
Request Header Query Body Raw +	Response Header Body Raw +
Key Value	1 {
Host 82.39.214.8:3001	2 {
Connection Keep-Alive	3 "images": [
Accept-Encoding gzip	4 "/photos/image-1782534246385.jpg"
User-Agent okhttp/4.12.0	5]
	6 }

GET 200 http://82.39.214.8:3001/api/photos/cats/5/photos	
Request Header Query Body Raw +	Response Header Body Raw +
Key Value	1 {
Host 82.39.214.8:3001	2 {
Connection Keep-Alive	3 "images": [
Accept-Encoding gzip	4 "/photos/image-1782534246385.jpg"
User-Agent okhttp/4.12.0	5]
	6 }

После отключения брейкпоинтов все вернулось в начальное состояние при последующем заходе на ту же карточку

Часть 5. Проблемы приложения

1. Некорректные HTTP-статусы при ошибках валидации

Title: Improper HTTP Status Codes: Возврат HTTP 200 ОК при ошибках бизнес-логики и валидации.

Severity: Medium

Preconditions: Доступ к API.

Steps to reproduce:

1. Попытаться добавить кота с именем, которое уже существует.
2. Отправить невалидные типы данных (например, число вместо строки).

Actual result: Сервер возвращает HTTP-статус 200 ОК, а информация об ошибке передается в теле ответа в кастомном поле errorDescription.

Expected result: Сервер должен использовать стандартные коды ответов: 400 Bad Request (для невалидных типов) и 409 Conflict (для дубликатов).

Evidence:

POST 200 http://82.39.214.8:3001/api/core/cats/add			
Request	Header	Query	Body Raw +
Key	Value		
Content-Type	application/json; charset=UTF-8		
Content-Length	67		
Host	82.39.214.8:3001		
Connection	Keep-Alive		
Accept-Encoding	gzip		
User-Agent	okhttp/4.12.0		
Response	Header	Body	Raw +
1		{	
2		"cats": [	
3		{	
4		"errorDescription": "Такое значение уже существует"	
5		}	
6		]	
7		}	

- Request: POST /api/core/cats/add (дубликат)
- Response: 200 OK {"cats": [{"errorDescription": "Такое значение уже существует"}]}

Recommendation: Настроить глобальный обработчик исключений (Exception Filter), чтобы при ошибках валидации и бизнес-логики возвращались корректные 4xx HTTP-статусы.

2. Нарушение бизнес-логики: Чередование лайка и дизлайка позволяет накручивать оба счетчика

Title: Business Logic Flaw: Возможность накрутки лайков и дизлайков путем чередования голосов.

Severity: High

Preconditions: Доступ к API, наличие любого catId.

Steps to reproduce:

1. Отправить запрос на лайк:
 POST /api/likes/cats/13/likes
 {
 "dislike": false,
 "like": true
 }
2. Затем отправить запрос на дизлайк:
 POST /api/likes/cats/13/likes
 {
 "dislike": true,
 "like": false
 }

}

3. Повторять последовательность "лайк → дизлайк → лайк → дизлайк" неограниченное количество раз.

4. Проверить значения счетчиков лайков и дизлайков.

Actual result:

При каждом переключении между лайком и дизлайком сервер увеличивает соответствующий счетчик, не уменьшая предыдущий и не проверяя существование голоса пользователя. В результате одновременно увеличиваются как количество лайков, так и количество дизлайков, что позволяет искусственно накручивать рейтинг объекта.

Expected result:

Для каждого пользователя должен существовать только один актуальный голос за объект. При смене реакции сервер должен обновлять существующую запись (заменять лайк на дизлайк и наоборот), не увеличивая общее количество голосов. Повторное голосование должно изменять текущий голос, а не создавать новый.

Evidence:

GET 200 http://82.39.214.8:3001/api/core/cats/get-by-id?id=13											
Request	Response										
<table><thead><tr><th>Key</th><th>Value</th></tr></thead><tbody><tr><td>Host</td><td>82.39.214.8:3001</td></tr><tr><td>Connection</td><td>Keep-Alive</td></tr><tr><td>Accept-Encoding</td><td>gzip</td></tr><tr><td>User-Agent</td><td>okhttp/4.12.0</td></tr></tbody></table>	Key	Value	Host	82.39.214.8:3001	Connection	Keep-Alive	Accept-Encoding	gzip	User-Agent	okhttp/4.12.0	<pre>1 { 2 "cat": { 3 "id": 13, 4 "name": "Котэ", 5 "description": "Алькотек", 6 "tags": "", 7 "gender": "male", 8 "likes": 6, 9 "dislikes": 0 10 } 11 }</pre>
Key	Value										
Host	82.39.214.8:3001										
Connection	Keep-Alive										
Accept-Encoding	gzip										
User-Agent	okhttp/4.12.0										

POST 200 http://82.39.214.8:3001/api/likes/cats/13/likes	
Request	Response
Header Query Body Raw +	Header Body Raw +
Key Value	1 {
Content-Type application/json; charset=UTF-8	2 "id": 13,
Content-Length 29	3 "name": "Котэ",
Host 82.39.214.8:3001	4 "description": "Алькотек",
Connection Keep-Alive	5 "tags": "",
Accept-Encoding gzip	6 "gender": "male",
User-Agent okhttp/4.12.0	7 "likes": 6,
	8 "dislikes": 1
	9 }

POST 200 http://82.39.214.8:3001/api/likes/cats/13/likes	
Request	Response
Header Query Body Raw +	Header Body Raw +
Key Value	1 {
Content-Type application/json; charset=UTF-8	2 "id": 13,
Content-Length 29	3 "name": "Котэ",
Host 82.39.214.8:3001	4 "description": "Алькотек",
Connection Keep-Alive	5 "tags": "",
Accept-Encoding gzip	6 "gender": "male",
User-Agent okhttp/4.12.0	7 "likes": 7,
	8 "dislikes": 1
	9 }

POST 200 http://82.39.214.8:3001/api/likes/cats/13/likes	
Request	Response
Header Query Body Raw +	Header Body Raw +
Key Value	1 {
Content-Type application/json; charset=UTF-8	2 "id": 13,
Content-Length 29	3 "name": "Котэ",
Host 82.39.214.8:3001	4 "description": "Алькотек",
Connection Keep-Alive	5 "tags": "",
Accept-Encoding gzip	6 "gender": "male",
User-Agent okhttp/4.12.0	7 "likes": 7,
	8 "dislikes": 2
	9 }

POST 200 http://82.39.214.8:3001/api/likes/cats/13/likes	
Request	Response
Header Query Body Raw +	Header Body Raw +
Key Value	1 {
Content-Type application/json; charset=UTF-8	2 "id": 13,
Content-Length 29	3 "name": "Котэ",
Host 82.39.214.8:3001	4 "description": "Алькотек",
Connection Keep-Alive	5 "tags": "",
Accept-Encoding gzip	6 "gender": "male",
User-Agent okhttp/4.12.0	7 "likes": 8,
	8 "dislikes": 2
	9 }

В результате после многократного чередования одновременно увеличиваются оба счетчика.

Recommendation:

Реализовать хранение единственного голоса пользователя для каждой записи (user_id, cat_id) с ограничением уникальности (UNIQUE(user_id, cat_id)). При изменении реакции выполнять обновление существующей записи вместо создания новой, а пересчет количества лайков и дизлайков производить на основе актуального состояния голосов.

3. Отсутствие Rate Limiting позволяет автоматизировать накрутку рейтинга

Title: Lack of Rate Limiting: Возможность автоматизированной эксплуатации уязвимости бизнес-логики.

Severity: Medium

Preconditions: Доступ к API.

Steps to reproduce:

1. Использовать сценарий из Проблемы 3 (чередование запросов лайка и дизлайка).
2. Настроить автоматическую отправку этих запросов с высокой частотой (например, с помощью Proxymap Repeat или скрипта).
3. Наблюдать за изменением счетчиков лайков и дизлайков.

Actual

result:

Сервер обрабатывает неограниченное количество запросов без каких-либо ограничений по частоте. Это позволяет автоматически и с высокой скоростью эксплуатировать уязвимость, описанную в Проблеме 3, значительно ускоряя накрутку лайков и дизлайков.

Expected

result:

При превышении допустимой частоты запросов (например, более 10 запросов в секунду с одного пользователя или IP-адреса) сервер должен возвращать ответ 429 Too Many Requests и временно ограничивать дальнейшие запросы.

Evidence:

Автоматическая отправка большого количества запросов не приводит к блокировке или ограничению клиента. Все запросы успешно обрабатываются,

что позволяет многократно эксплуатировать уязвимость, описанную в Проблеме 2.

Recommendation:

Внедрить механизм Rate Limiting (например, с использованием Redis и express-rate-limit либо на уровне API Gateway/Nginx). Ограничение должно применяться как минимум к эндпоинтам голосования, чтобы предотвратить автоматизированную эксплуатацию бизнес-логических уязвимостей и снизить риск DoS-атак.

4. Нарушение контроля доступа: Возможность изменения описания чужого кота

Title: Broken Object Level Authorization (BOLA): Отсутствие проверки прав доступа при изменении описания кота.

Severity: Critical

Preconditions: Авторизованный пользователь, знание catId любого кота в системе.

Steps to reproduce:

1. Перехватить запрос изменения описания кота в Proxuman.
2. Изменить значение параметра catId на идентификатор кота, принадлежащего другому пользователю.
3. Отправить запрос:
4. POST /api/core/cats/save-description
5. {
6. "catDescription": "Новый текст",
7. "catId": 21
8. }
9. Проверить описание указанного кота.

Actual

result:

Сервер успешно изменяет описание чужого кота и возвращает ответ HTTP 200

ОК с обновленными данными, несмотря на отсутствие прав на редактирование данного объекта.

Expected

result:

Перед выполнением операции сервер должен проверить, что текущий пользователь является владельцем кота (либо обладает соответствующими правами). При отсутствии доступа сервер должен вернуть HTTP 403 Forbidden (или 404 Not Found, чтобы скрыть существование объекта).

Evidence:

The image displays two screenshots of a REST client interface, likely Swagger UI, showing a POST request to the endpoint `http://82.39.214.8:3001/api/core/cats/save-description`.

Top Screenshot:

- Method: POST
- Status: Active
- Request Body (JSON):

```
{  "catDescription": "Алькотек",  "catId": 13}
```

Bottom Screenshot:

- Method: POST
- Status: Active
- Request Body (JSON):

```
{  "catDescription": "Алькотек не алькотек",  "catId": 13}
```


POST 200 http://82.39.214.8:3001/api/core/cats/save-description	
Request	Response
Header Query Body Raw +	Header Body Raw +
JSON	JSON
<pre> 1 { 2 "catDescription": "Алькотек не алькотек", 3 "catId": 13 4 }</pre>	<pre> 1 { 2 "id": 13, 3 "name": "Котэ", 4 "description": "Алькотек не алькотек", 5 "tags": "", 6 "gender": "male", 7 "likes": 8, 8 "dislikes": 2 9 }</pre>

Финальный результат

GET 200 http://82.39.214.8:3001/api/core/cats/get-by-id?id=13	
Request	Response
Header Query Body Raw +	Header Body Raw +
Empty Body	JSON
	<pre> 1 { 2 "cat": { 3 "id": 13, 4 "name": "Котэ", 5 "description": "Алькотек не алькотек", 6 "tags": "", 7 "gender": "male", 8 "likes": 8, 9 "dislikes": 2 10 } 11 }</pre>

- Request:
- {
- "catDescription": "Новый текст",
- "catId": 13
- }
- Response: HTTP 200 OK, описание кота успешно изменено.

Recommendation:

Реализовать проверку авторизации на уровне бизнес-логики. Перед обновлением описания необходимо сравнивать идентификатор пользователя из токена/сессии с владельцем объекта (ownerId) для указанного catId. Выполнять операцию изменения данных только при наличии соответствующих прав доступа.

5. Отсутствие обязательной аутентификации при выполнении критических операций

Title: Missing Authentication: Критические API-эндпоинты доступны без аутентификации.

Severity: Critical

Preconditions: Доступ к API.

Steps to reproduce:

1. Отправить запрос на выполнение операции, изменяющей данные (например, создание кота, голосование, изменение описания или удаление).
2. Не передавать заголовки Authorization, Cookie, JWT-токен или иные учетные данные.
3. Проверить результат выполнения запроса.

Actual result:

Сервер успешно выполняет операции, изменяющие данные, несмотря на отсутствие какой-либо аутентификации пользователя. Запросы обрабатываются как легитимные и возвращают успешный ответ (HTTP 200 OK).

Expected result:

Все эндпоинты, выполняющие создание, изменение или удаление данных, должны быть доступны только аутентифицированным пользователям. При отсутствии действительных учетных данных сервер должен возвращать HTTP 401 Unauthorized и не выполнять запрошенную операцию.

Evidence:

Анализ HTTP-запросов, перехваченных в Proxman, показывает отсутствие заголовков аутентификации (Authorization, Cookie, JWT и др.), при этом сервер успешно выполняет операции добавления, изменения, удаления и голосования.

Recommendation:

Внедрить механизм аутентификации (например, JWT или механизм сессий) и применить middleware проверки аутентификации ко всем эндпоинтам, изменяющим состояние системы. Исключение могут составлять только публичные операции, такие как получение списка котов или просмотр рейтинга.

6. Публичная доступность Swagger UI без аутентификации

Title: Security Misconfiguration: Swagger UI и OpenAPI-спецификация доступны без аутентификации.

Severity: Medium

Preconditions: Доступ к приложению по сети.

Steps to reproduce:

1. Перейти по адресу Swagger UI (например, /swagger, /swagger-ui, /api-docs или другому используемому маршруту).
2. Не выполнять аутентификацию и не передавать заголовок Authorization.
3. Изучить доступную документацию API.

Actual

result:

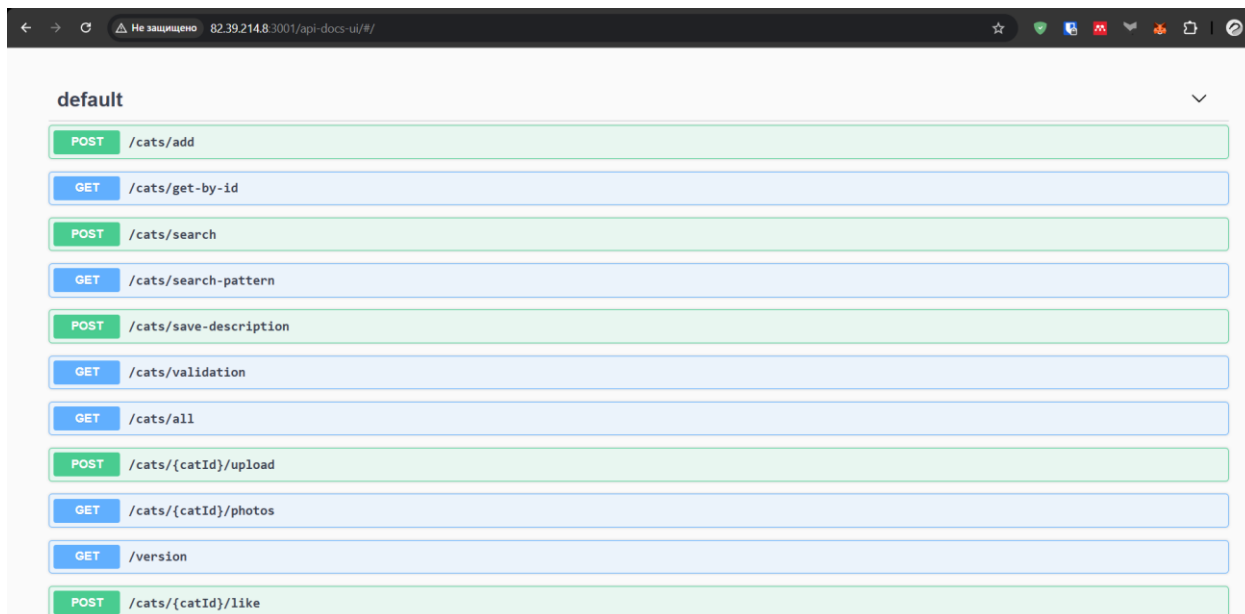
Swagger UI и OpenAPI-спецификация доступны любому пользователю без прохождения аутентификации. Документация содержит полный перечень API-эндпоинтов, поддерживаемые HTTP-методы, параметры запросов, модели данных и примеры вызовов.

Expected

result:

Документация API не должна быть доступна неавторизованным пользователям в производственной среде. При обращении к Swagger UI сервер должен требовать аутентификацию (возвращать 401 Unauthorized или 403 Forbidden) либо полностью отключать документацию на production.

Evidence:



При переходе на страницу Swagger UI без передачи каких-либо учетных данных отображается полная документация API, включая описание всех доступных эндпоинтов и схем запросов.

Recommendation:

Ограничить доступ к Swagger UI и OpenAPI-спецификации одним из следующих способов:

- требовать аутентификацию для доступа к документации;
- ограничить доступ по IP или VPN для внутренних пользователей;
- полностью отключать Swagger UI в production-среде, оставляя его доступным только в средах разработки и тестирования.

Вывод

В результате проведенного тестирования были выявлены несколько критических уязвимостей API, связанных с отсутствием должного контроля безопасности на стороне сервера. Анализ показал, что клиентское приложение не может рассматриваться как доверенная среда, поскольку любой пользователь способен перехватывать, изменять и повторно отправлять HTTP-запросы с помощью специализированных инструментов.

Наиболее серьезной проблемой является отсутствие обязательной аутентификации для критических операций. API позволяет выполнять создание, изменение и удаление данных без подтверждения личности пользователя. В сочетании с отсутствием проверки прав доступа это приводит к возможности изменять данные, принадлежащие другим пользователям, что является нарушением принципов безопасного управления доступом.

Дополнительно были выявлены недостатки реализации бизнес-логики. Сервер не контролирует уникальность голосов пользователя, вследствие чего становится возможной искусственная накрутка рейтинга путем чередования лайков и дизлайков. Отсутствие механизма ограничения частоты запросов (Rate Limiting) значительно упрощает автоматизацию подобных атак и позволяет злоумышленнику выполнять их с высокой скоростью.

Еще одним недостатком является публичная доступность Swagger UI и OpenAPI-документации. Наличие открытой документации существенно облегчает изучение структуры API, доступных методов и форматов запросов, что снижает порог для проведения дальнейших атак.

Таким образом, проведенное тестирование показывает, что безопасность веб-сервиса должна обеспечиваться исключительно серверной частью приложения. Клиентский интерфейс может повышать удобство использования, однако не должен использоваться в качестве механизма защиты.

Для устранения выявленных недостатков рекомендуется:

- внедрить обязательную аутентификацию для всех операций, изменяющих данные;

- реализовать проверку прав доступа к каждому объекту перед выполнением операций изменения или удаления;
- обеспечить контроль бизнес-логики голосования, исключив возможность создания нескольких голосов одним пользователем;
- внедрить механизм ограничения частоты запросов для защиты от автоматизированных атак;
- ограничить доступ к Swagger UI и другой служебной документации в производственной среде;
- придерживаться принципа *server-side security*, при котором все критически важные проверки выполняются исключительно на стороне сервера.

Реализация перечисленных мер позволит существенно повысить уровень защищенности приложения, исключить возможность эксплуатации обнаруженных уязвимостей и привести архитектуру API в соответствие с современными требованиями информационной безопасности.